

ALGORITHM DEVELOPMENT FOR EACH MODULE

1.Pre-Entry Slot Availability Display Algorithm

ALGORITHM CheckSlotAvailability

INPUT: lot_id

OUTPUT: status_message, available_slots

1. Query Redis cache for 'lot_id:available_slots'

2. IF cache_miss THEN

 Fetch available_slots from 'parking_lots' database table

 Set cache 'lot_id:available_slots' with TTL = 60 seconds

ENDIF

3. IF available_slots > 0 THEN

 Set display_text = "WELCOME: " + available_slots + " SLOTS AVAILABLE"

 Set gate_status = OPEN_ALLOWED

ELSE

 Set display_text = "LOT FULL - NO SPACES AVAILABLE"

 Set gate_status = ENTRY_BLOCKED

ENDIF

4. Render display_text on external visual LED screen

5. RETURN gate_status, available_slots

END ALGORITHM

2.Vehicle Arrival and Entry Algorithm

ALGORITHM RecordVehicleEntry

INPUT: plate_number, lot_id

OUTPUT: entry_status, invoice_ticket

1. Normalize plate_number = TRIM(UPPER(plate_number))
2. LOCK critical_section
3. Read current available_slots for lot_id
4. IF available_slots <= 0 THEN
 - UNLOCK critical_section
 - RETURN "ENTRY DENIED: Parking Lot Full"
- ENDIF
5. Check active_vehicles dictionary for plate_number
6. IF plate_number EXISTS in active_vehicles THEN
 - UNLOCK critical_section
 - RETURN "ENTRY DENIED: Duplicate Vehicle Record Active"
- ENDIF
7. Capture current_timestamp = GET_CURRENT_DATETIME()
8. INSERT INTO parking_sessions (lot_id, plate_number, entry_time, status)
 - VALUES (lot_id, plate_number, current_timestamp, 'ACTIVE')
9. DECREMENT available_slots BY 1 in database and Redis cache
10. UNLOCK critical_section
11. TRIGGER entry_barrier_actuator(SIGNAL_OPEN)
12. RETURN "ENTRY APPROVED: Barrier Opened", current_timestamp

END ALGORITHM

3.Vehicle exit and Fee Calculation Algorithm

ALGORITHM CalculateExitFee

INPUT: plate_number, lot_id

OUTPUT: duration_hours, amount_due, session_id

1. Normalize plate_number = TRIM(UPPER(plate_number))
2. FETCH active record from parking_sessions WHERE plate_number AND status = 'ACTIVE'
3. IF record NOT FOUND THEN
 - RAISE EXCEPTION "ERROR: No active arrival record found for vehicle"
 - ENDIF
4. Set exit_timestamp = GET_CURRENT_DATETIME()
5. Calculate elapsed_seconds = exit_timestamp - entry_time
6. Convert elapsed_seconds to hours:
 - duration_hours = CEIL(elapsed_seconds / 3600)
 - IF duration_hours < 1 THEN duration_hours = 1
7. Fetch hourly_rate from parking_lots WHERE lot_id = session.lot_id
8. Calculate amount_due = duration_hours * hourly_rate
9. RETURN session_id, duration_hours, amount_due, exit_timestamp

END ALGORITHM

4.Payment Verification and Barrier Release Algorithm

ALGORITHM ProcessPaymentAndReleaseExit

INPUT: session_id, amount_paid, payment_method

OUTPUT: exit_status

1. FETCH session record by session_id
2. IF amount_paid < session.amount_due THEN
 - RETURN "PAYMENT FAILED: Insufficient Amount Paid"
 - ENDIF

3. BEGIN TRANSACTION

4. INSERT INTO payments (session_id, amount_paid, payment_time, payment_method)

VALUES (session_id, amount_paid, GET_CURRENT_DATETIME(), payment_method)

5. UPDATE parking_sessions

SET exit_time = GET_CURRENT_DATETIME(), status = 'COMPLETED'

WHERE session_id = session_id

6. INCREMENT available_slots BY 1 in 'parking_lots' AND Redis cache

7. COMMIT TRANSACTION

8. TRIGGER exit_barrier_actuator(SIGNAL_OPEN)

9. RETURN "PAYMENT SUCCESSFUL: Exit Barrier Opened"

END ALGORITHM